

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING**

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment :	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	JAYAKRISHNA.V	Reg. No.:	192421259
Section / Batch:	I	Date:	27.07.26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q 30

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I JAYAKRISHNA.V 192421259 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student JAYAKRISHNA.V

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding ; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q29. Sorting Techniques – Selection Sort vs Bubble Sort

1. Understanding the Problem

Sorting is one of the most fundamental operations in computer science. It is the process of arranging data in a specific order, either ascending or descending, to make the data easier to search, analyze, and process. Many real-world applications such as banking systems, student databases, online shopping websites, hospital management systems, and library management systems use sorting algorithms to organize records efficiently. In this problem, we are given the following unsorted dataset:

$$A = [28, 16, 35, 11, 19]$$

The objective is to sort the above dataset in **ascending order** using two different Brute Force sorting algorithms:

- **Selection Sort**
- **Bubble Sort**

The problem requires us to perform the following tasks:

- Clearly understand how both algorithms work.
- Apply each algorithm to the given dataset.
- Show the intermediate array after every pass.
- Count the total number of comparisons.
- Count the total number of swaps.
- Analyze the time complexity of both algorithms.
- Compare their overall performance.
- Identify which algorithm is more suitable for the given dataset with proper justification.

The expected sorted array is:

Final Sorted Array

[11, 16, 19, 28, 35]

2. Problem Interpretation

The given dataset contains **five unsorted integer values**. Since the elements are not arranged in ascending order, both sorting algorithms must repeatedly compare elements and perform swap operations until the array becomes completely sorted.

Although Selection Sort and Bubble Sort both belong to the Brute Force family and have the same worst-case time complexity of $O(n^2)$, their working methods are different.

- **Selection Sort** repeatedly selects the smallest element from the unsorted portion and places it in the correct position.
- **Bubble Sort** repeatedly compares adjacent elements and swaps them whenever they are in the wrong order.

Therefore, this problem is not only about obtaining the sorted array but also about studying the efficiency of each algorithm by analyzing their comparisons, swaps, and execution process.

4. Selection Sort

Definition

Selection Sort is a simple comparison-based sorting algorithm. During each pass, it selects the smallest element

from the unsorted portion of the array and exchanges it with the first unsorted element. After every pass, one more element reaches its correct sorted position.

Unlike Bubble Sort, Selection Sort performs only one swap at the end of each pass, making it efficient when the cost of swapping is high.

Working Principle

The working of Selection Sort can be explained through the following steps:

1. Assume the first element is the minimum.
2. Compare it with every remaining element in the array.
3. If a smaller element is found, update the minimum.
4. After completing the comparisons, swap the smallest element with the first unsorted element.
5. Move to the next position and repeat the same process.
6. Continue until all elements are arranged in ascending order.

This process gradually increases the sorted portion of the array from left to right.

Algorithm

SelectionSort(A, n)

for i = 0 to n-2

 min = i

 for j = i+1 to n-1

 if $A[j] < A[\text{min}]$

 min = j

 Swap(A[i], A[min])

5. Step-by-Step Execution of Selection Sort

Initial Dataset

Index 0 1 2 3 4

Value 28 16 35 11 19

Current Array:

[28, 16, 35, 11, 19]

Pass 1

The algorithm starts from the first position and searches for the smallest element.

Comparisons:

- Compare 28 and 16 → Minimum = 16

- Compare 16 and 35 → Minimum remains 16
- Compare 16 and 11 → Minimum = 11
- Compare 11 and 19 → Minimum remains 11

Smallest element found = **11**

Swap **28** with **11**

Array after Pass 1

[11, 16, 35, 28, 19]

Comparisons = **4**

Swaps = **1**

Analysis:

After the first pass, the smallest element (**11**) has been placed in its correct position. The first position is now sorted, while the remaining elements are still unsorted.

Pass 2

Now the algorithm considers only the remaining unsorted elements.

Current unsorted portion:

16 35 28 19

Comparisons:

- Compare 16 and 35
- Compare 16 and 28
- Compare 16 and 19

The smallest element is already **16**, so no swap is required.

Array after Pass 2

[11, 16, 35, 28, 19]

Comparisons = **3**

Swaps = **0**

Analysis:

Since 16 is already the smallest element in the remaining unsorted portion, the algorithm does not perform any swap. This shows that Selection Sort avoids unnecessary swapping whenever the minimum element is already in the correct position.

5. Step-by-Step Execution of Selection Sort (Continued)

Pass 3

After completing Pass 2, the array is:

[11, 16, 35, 28, 19]

The first two elements (**11** and **16**) are already in their correct positions. Therefore, the algorithm searches for the smallest element only in the remaining unsorted portion.

Current unsorted elements:

35 28 19

Comparisons

- Compare **35** and **28** → Minimum becomes **28**
- Compare **28** and **19** → Minimum becomes **19**

The smallest element in the remaining unsorted portion is **19**.

Therefore, **35** and **19** are swapped.

Array after Pass 3

[11, 16, 19, 28, 35]

Comparisons = 2

Swaps = 1

Analysis

At the end of the third pass, the third smallest element (**19**) reaches its correct position. Now the first four elements are arranged correctly, leaving only one element unsorted.

Pass 4

Current array:

[11, 16, 19, 28, 35]

Remaining unsorted elements:

28 35

Comparison

- Compare **28** and **35**

Since **28** is already smaller than **35**, no swap is required.

Final Sorted Array

[11, 16, 19, 28, 35]

Comparisons = 1

Swaps = 0

Analysis

After the fourth pass, every element is in its correct ascending order. The sorting process is successfully completed.

6. Summary of Selection Sort

Pass	Array after Pass	Comparisons	Swaps
1	[11, 16, 35, 28, 19]	4	1
2	[11, 16, 35, 28, 19]	3	0
3	[11, 16, 19, 28, 35]	2	1
4	[11, 16, 19, 28, 35]	1	0
Total	[11, 16, 19, 28, 35]	10	2

Observation

From the above execution, Selection Sort required:

- **Total Comparisons = 10**
- **Total Swaps = 2**

Although the algorithm compared elements many times, it performed only two swaps because swapping is done only after identifying the smallest element in each pass.

7. Bubble Sort

Definition

Bubble Sort is a simple comparison-based sorting algorithm that repeatedly compares two adjacent elements. Whenever two adjacent elements are found in the wrong order, they are exchanged. During each pass, the largest unsorted element gradually moves towards the end of the array, similar to the way a bubble rises to the surface of water.

Because it swaps adjacent elements frequently, Bubble Sort usually performs more swap operations than Selection Sort.

Algorithm

BubbleSort(A, n)

for i = 0 to n-2

 for j = 0 to n-i-2

 if A[j] > A[j+1]

 Swap(A[j], A[j+1])

8. Step-by-Step Execution of Bubble Sort

Initial Dataset

Index 0 1 2 3 4

Value 28 16 35 11 19

Current Array:

[28, 16, 35, 11, 19]

Pass 1

The algorithm compares adjacent elements one by one.

Comparisons

- Compare **28** and **16** → Swap

16 28 35 11 19

- Compare **28** and **35** → No Swap

16 28 35 11 19

- Compare **35** and **11** → Swap

16 28 11 35 19

- Compare **35** and **19** → Swap

16 28 11 19 35

Array after Pass 1

[16, 28, 11, 19, 35]

Comparisons = 4

Swaps = 3

Analysis

At the end of the first pass, the largest element (**35**) has moved to the last position. However, the remaining elements are still unsorted, so another pass is required.

8. Step-by-Step Execution of Bubble Sort (Continued)

Pass 2

After completing Pass 1, the array becomes:

[16, 28, 11, 19, 35]

Since the largest element (**35**) has already reached its correct position, it is not considered in this pass.

The algorithm now compares only the remaining four elements.

Comparison 1

Compare **16** and **28**

Since **16 < 28**, both elements are already in the correct order.

No Swap

Array remains

16 28 11 19 35

Comparison 2

Compare **28** and **11**

Since **28 > 11**, swap both elements.

Array becomes

16 11 28 19 35

Comparison 3

Compare **28** and **19**

Since **28 > 19**, swap both elements.

Array becomes

16 11 19 28 35

Array after Pass 2

[16, 11, 19, 28, 35]

Comparisons = 3

Swaps = 2

Analysis

At the end of the second pass, the second largest element (**28**) reaches its correct position.
Now the last two elements are completely sorted.

Pass 3

Current Array

[16, 11, 19, 28, 35]

Now only the first three elements are unsorted.

Comparison 1

Compare **16** and **11**

Since **16 > 11**, swap them.

Array becomes

11 16 19 28 35

Comparison 2

Compare **16** and **19**

Since **16 < 19**

No swap is required.

Array remains

11 16 19 28 35

Array after Pass 3

[11, 16, 19, 28, 35]

Comparisons = **2**

Swaps = **1**

Analysis

At the end of the third pass, the third largest element has reached its correct position.
Almost the entire array is now sorted.

Pass 4

Current Array

[11, 16, 19, 28, 35]

Only one comparison is required.

Comparison

Compare

11 and **16**

Since **11 < 16**

No swap is required.

Array remains unchanged.

[11, 16, 19, 28, 35]

Comparisons = **1**

Swaps = **0**

Final Sorted Array

[11, 16, 19, 28, 35]

The Bubble Sort algorithm has successfully arranged the given dataset in ascending order after four passes.

9. Summary of Bubble Sort

Pass	Array after Pass	Comparisons	Swaps
Pass 1	[16, 28, 11, 19, 35]	4	3
Pass 2	[16, 11, 19, 28, 35]	3	2
Pass 3	[11, 16, 19, 28, 35]	2	1
Pass 4	[11, 16, 19, 28, 35]	1	0
Total	[11, 16, 19, 28, 35]	10	6

10. Analysis of Bubble Sort

From the above execution, Bubble Sort successfully sorted the given dataset into ascending order.

The algorithm performed **10 comparisons** and **6 swap operations**.

Bubble Sort repeatedly exchanged adjacent elements whenever they were found in the wrong order. Because of this repeated swapping, Bubble Sort required more data movement than Selection Sort.

Although Bubble Sort is very simple to understand and implement, it becomes inefficient when the dataset size increases because the number of swap operations also increases significantly.

For small datasets like the given example, Bubble Sort produces the correct result, but it performs more swaps than Selection Sort.

11. Observation

The final sorted array obtained using Bubble Sort is

[11, 16, 19, 28, 35]

The total number of operations performed is:

- **Total Comparisons = 10**
- **Total Swaps = 6**

From the above execution, it is evident that Bubble Sort performs the same number of comparisons as Selection Sort for this dataset but requires a much larger number of swap operations.

This indicates that Bubble Sort spends more time exchanging elements, making it comparatively less efficient for this particular dataset.

12. Time Complexity Analysis

Time complexity is used to measure the amount of time required by an algorithm as the input size increases. It helps us understand the efficiency of an algorithm and compare different algorithms based on their performance.

Time Complexity of Selection Sort

Selection Sort searches for the smallest element in the unsorted portion during every pass. Even if the array is already sorted, the algorithm still compares every remaining element to find the minimum value. Therefore, the number of comparisons remains almost the same for all input cases.

Case	Time Complexity	Explanation
Best Case	$O(n^2)$	All comparisons are performed even if the array is already sorted.
Average Case	$O(n^2)$	Every pass searches for the minimum element from the remaining unsorted portion.
Worst Case	$O(n^2)$	Maximum comparisons are required when the array is in reverse order.

Space Complexity: $O(1)$

Selection Sort requires only a few extra variables for storing the current minimum index. Therefore, it is considered an in-place sorting algorithm.

Time Complexity of Bubble Sort

Bubble Sort repeatedly compares adjacent elements and exchanges them whenever they are in the wrong order. The number of passes depends on the arrangement of the elements. In a simple Bubble Sort implementation (without optimization), all passes are executed even if the array becomes sorted before the final pass.

Case	Time Complexity	Explanation
Best Case	$O(n)$ (<i>Optimized Bubble Sort</i>) or $O(n^2)$ (<i>Basic Bubble Sort</i>)	If a swap flag is used, the algorithm stops after one pass when the array is already sorted.
Average Case	$O(n^2)$	Multiple comparisons and swaps are required to arrange the elements correctly.
Worst Case	$O(n^2)$	Maximum comparisons and swaps occur when the array is in reverse order.

Space Complexity: $O(1)$

Bubble Sort is also an in-place sorting algorithm because it requires only constant extra memory.

13. Comparison Between Selection Sort and Bubble Sort

Feature	Selection Sort	Bubble Sort
Sorting Method	Selects the smallest element and places it in the correct position	Compares adjacent elements and swaps them
Total Comparisons	10	10
Total Swaps	2	6
Time Complexity	$O(n^2)$	$O(n^2)$

Feature	Selection Sort	Bubble Sort
Space Complexity	$O(1)$	$O(1)$
Number of Swaps	Less	More
Stability	Not Stable	Stable
Adaptive Nature	Not Adaptive	Adaptive (Optimized Version)
Suitable for Large Data	No	No
Suitable for Small Data	Yes	Yes
Overall Efficiency for this Dataset	Better	Less Efficient

14. Performance Analysis

The execution of both algorithms on the dataset

[28, 16, 35, 11, 19]

shows that both methods successfully arranged the elements in ascending order.

The final sorted array obtained by both algorithms is:

[11, 16, 19, 28, 35]

Although both algorithms required **10 comparisons**, there is a significant difference in the number of swap operations.

- **Selection Sort** performed only **2 swaps** because it exchanges elements only after finding the smallest value during each pass.
- **Bubble Sort** performed **6 swaps** because it exchanges adjacent elements repeatedly throughout the sorting process.

Since swap operations involve moving data in memory, a larger number of swaps generally results in additional processing time. Therefore, Selection Sort completed the sorting process with less data movement than Bubble Sort.

This analysis shows that even when two algorithms have the same time complexity, their practical performance may differ because of the number of swap operations performed.

15. Interpretation of Results

The comparison clearly shows that both Selection Sort and Bubble Sort are capable of sorting the given dataset correctly. However, their internal working mechanisms are different.

Selection Sort first identifies the smallest element and places it directly in its correct position. As a result, unnecessary swap operations are avoided. Bubble Sort, on the other hand, continuously exchanges adjacent elements, leading to a larger number of swaps.

For the given dataset:

- **Selection Sort**
 - Comparisons = **10**
 - Swaps = **2**
- **Bubble Sort**
 - Comparisons = **10**

- Swaps = 6

Since the number of comparisons is the same but the number of swaps is much lower in Selection Sort, it performs fewer data movements. This makes it more efficient for this particular dataset.

16. Justification

Based on the above analysis, **Selection Sort is the preferred algorithm for the given dataset.**

The reasons are:

- It required fewer swap operations.
- It completed the sorting process with less data movement.
- It successfully arranged all elements in ascending order.
- It used the same amount of memory as Bubble Sort.
- It achieved the same final result while performing fewer exchanges.

Although both algorithms have a worst-case time complexity of $O(n^2)$, Selection Sort proved to be more efficient for this dataset because it minimized the number of swaps.

17. Conclusion

Selection Sort and Bubble Sort are two simple Brute Force sorting algorithms used to arrange data in ascending or descending order.

In this problem, both algorithms successfully sorted the dataset [28, 16, 35, 11, 19] into [11, 16, 19, 28, 35].

The detailed pass-by-pass analysis showed that both algorithms required **10 comparisons**. However, Selection Sort performed only **2 swaps**, whereas Bubble Sort performed **6 swaps**.

Therefore, Selection Sort demonstrated better performance for the given dataset because it reduced unnecessary data movement while producing the same sorted output.

Hence, Selection Sort is the more suitable algorithm for this dataset, whereas Bubble Sort, although simple and easy to understand, performs more swap operations and is comparatively less efficient in this case.